[NOMBRE DE LA INSTITUCIÓN]

[Facultad / Carrera]

Desarrollo de Aplicaciones Móviles



# NexoShop

## Integración de APIs REST públicas

Parte 3 · Módulo de divisas en tiempo real con la API Frankfurter

| | |
|---|---|
| **Estudiante** | Jefferson Ricardo Vera Macas |
| **Asignatura** | Desarrollo de Aplicaciones Móviles |
| **Docente** | [Nombre del docente] |
| **Repositorio** | https://github.com/jeffvera281996/catalogo-productos-flutter-p3 |
| **Fecha de entrega** | [dd/mm/2026] |

# Sección I: Selección y descripción de la API pública

| Dato | Detalle |
|---|---|
| **Nombre** | Frankfurter: tasas de cambio de referencia del Banco Central Europeo (BCE) |
| **Documentación** | https://frankfurter.dev |
| **Endpoint usado** | GET https://api.frankfurter.dev/v1/latest?base=USD |
| **Parámetro** | base: moneda de referencia (USD, EUR, GBP o MXN en la app) |
| **Autenticación** | Ninguna: gratuita, sin API key, sin sesión ni token |
| **Formato** | JSON, actualizado cada día hábil (~16:00 CET) |

## Justificación de la elección

NexoShop es un catálogo de productos con precios en dólares. Ecuador usa el dólar, pero la tienda también atiende clientes que piensan en euros, pesos mexicanos o reales. La nueva pantalla **Divisas** convierte cualquier monto a 30 monedas con las tasas del día.

- **Encaja con el negocio:** no es una función aislada; ayuda a decidir una compra.
- **Datos en tiempo real:** las tasas cambian a diario, lo que justifica el refresco manual (pull-to-refresh).
- **Cumple las restricciones:** es REST, pública y gratuita, y no exige registro ni autenticación.
- **Fuente confiable:** los datos provienen del BCE, una fuente oficial.

## Estructura del JSON consumido

**Respuesta de GET /v1/latest?base=USD (valores ilustrativos)**

```
{
  "amount": 1.0,                // monto de referencia
  "base": "USD",                // moneda base solicitada
  "date": "2026-11-06",         // fecha de publicación del BCE
  "rates": {                    // mapa dinámico: código ISO 4217 -> tasa
    "AUD": 1.52,
    "EUR": 0.8712,
    "JPY": 152.1,
    "MXN": 18.43
    // ... 30 monedas en total
  }
}
```

El reto del mapeo es `rates`: no es una lista sino un objeto cuyas claves cambian. El modelo `TasasCambio` lo transforma en una `List<TasaCambio>` tipada y ordenada. Los nombres en español y las banderas se resuelven localmente porque la API solo devuelve códigos.

# Sección II: Arquitectura y manejo de estados

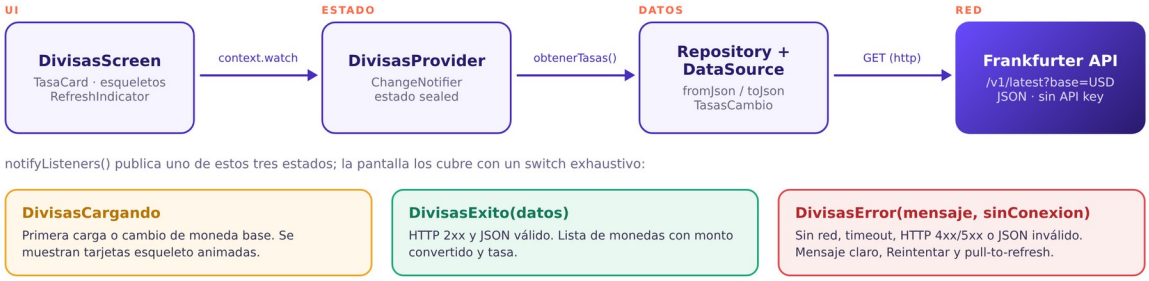



*Figura 1. Capas del módulo Divisas y estados de la petición.*

El módulo reutiliza la arquitectura por capas de la Actividad 2 y el patrón **Provider + ChangeNotifier**. Cada capa solo conoce a la siguiente:

| Capa | Clase | Responsabilidad |
|---|---|---|
| **UI** | DivisasScreen, TasaCard | Dibuja según el estado. No conoce HTTP ni JSON. |
| **Estado** | DivisasProvider | Ejecuta la petición, traduce el resultado a un estado y avisa a la UI con notifyListeners(). |
| **Repositorio** | DivisasRepository | Punto de acceso a los datos. Sin caché, a propósito: una tasa vieja mostrada como actual sería engañosa. |
| **Fuente remota** | DivisasRemoteDataSource | Arma la URL y mapea el JSON al modelo. |
| **Red** | ApiClient (paquete http) | GET asíncrono, timeout de 8 s y traducción de errores técnicos a excepciones de dominio. |

## Los tres estados de la petición

El estado es una jerarquía `sealed` de Dart 3. Con ella el compilador obliga a la pantalla a manejar los tres casos; si falta uno, el proyecto no compila.

| Estado | Cuándo ocurre | Qué ve el usuario |
|---|---|---|
| **DivisasCargando** | Primera petición o cambio de moneda base | Tarjetas "esqueleto" que parpadean |
| **DivisasExito(datos)** | HTTP 2xx y JSON válido | Lista de tarjetas: bandera, monto convertido y tasa unitaria |
| **DivisasError(mensaje, sinConexion)** | Sin red, timeout, HTTP 4xx/5xx o JSON inválido | Ícono, mensaje claro, botón Reintentar y pull-to-refresh |

### Decisiones de diseño

- **Errores con significado.** ApiClient convierte SocketException, TimeoutException y ClientException en SinConexionException, y los códigos HTTP en ServidorException. El Provider distingue "no hay internet" (el usuario puede resolverlo) de "falló el servidor".
- **Refresco sin parpadeo.** Con pull-to-refresh y datos visibles, el Provider no vuelve a "Cargando": el RefreshIndicator ya muestra el progreso y la lista se queda en pantalla.
- **Respuestas obsoletas.** Cada petición recibe un número. Si el usuario cambia de moneda base antes de que llegue la respuesta anterior, esa respuesta se descarta para no mezclar tasas de otra base.
- **Cálculo local.** Cambiar el monto no genera peticiones: se multiplica por la tasa ya descargada.

## Sección III: Extractos de código clave

### 3.1 Consumo HTTP y mapeo del modelo JSON

**Petición GET y traducción de errores**

```
// lib/data/datasources/divisas_remote_datasource.dart
class DivisasRemoteDataSource {
  static const String baseUrl = 'https://api.frankfurter.dev/v1';
  final ApiClient _api;

  Future<TasasCambio> obtenerTasas({required String base}) async {
    final json = await _api.get('/latest?base=$base');   // GET asíncrono
    try {
      return TasasCambio.fromJson(json);
    } on TypeError {
      throw const DatosInvalidosException();   // campo faltante o de otro tipo
    } on FormatException {
      throw const DatosInvalidosException();   // fecha mal formada
    }
  }
}

// lib/data/datasources/api_client.dart (extracto)
Future<Map<String, dynamic>> _ejecutar(Future<http.Response> Function() peticion) async {
  final http.Response respuesta;
  try {
    respuesta = await peticion().timeout(timeout);           // 8 segundos
  } on SocketException {
    throw const SinConexionException();
  } on TimeoutException {
    throw const SinConexionException('El servidor tardo demasiado.');
  }
  if (respuesta.statusCode < 200 || respuesta.statusCode >= 300) {
    throw ServidorException(respuesta.statusCode);
  }
  return jsonDecode(respuesta.body) as Map<String, dynamic>;
}
```

**Modelo con fromJson / toJson**

```
// lib/data/models/tasa_cambio.dart
class TasasCambio {
  final String base;
  final DateTime fecha;
  final double monto;
  final List<TasaCambio> tasas;

  factory TasasCambio.fromJson(Map<String, dynamic> json) {
    final rates = json['rates'] as Map<String, dynamic>;
    final tasas = rates.entries                                  // mapa -> lista tipada
        .map((e) => TasaCambio(codigo: e.key, valor: (e.value as num).toDouble()))
        .toList()
      ..sort((a, b) => Monedas.compararCodigos(a.codigo, b.codigo));
    return TasasCambio(
      base: json['base'] as String,
      fecha: DateTime.parse(json['date'] as String),
```

```
      monto: (json['amount'] as num).toDouble(),
      tasas: List.unmodifiable(tasas),
    );
  }

  Map<String, dynamic> toJson() => {
    'amount': monto,
    'base': base,
    'date': _fechaIso(fecha),
    'rates': {for (final t in tasas) t.codigo: t.valor},
  };
}
```

## 3.2 Manejo de estados en la interfaz

### Provider: estados sealed y lógica de carga

```
// lib/providers/divisas_provider.dart
sealed class DivisasEstado { const DivisasEstado(); }
final class DivisasCargando extends DivisasEstado { const DivisasCargando(); }
final class DivisasExito extends DivisasEstado { const DivisasExito(this.datos); final
TasasCambio datos; }
final class DivisasError extends DivisasEstado {
  const DivisasError(this.mensaje, {this.sinConexion = false});
  final String mensaje;
  final bool sinConexion;
}

Future<void> cargar({bool refrescar = false}) async {
  final idPeticion = ++_peticionActual;
  _cargando = true;
  if (!(refrescar && _estado is DivisasExito)) _estado = const DivisasCargando();
  notifyListeners();

  DivisasEstado nuevo;
  try {
    nuevo = DivisasExito(await _repositorio.obtenerTasas(base: _base));
  } on SinConexionException catch (e) {
    nuevo = DivisasError(e.mensaje, sinConexion: true);
  } on AppException catch (e) {
    nuevo = DivisasError(e.mensaje);
  } catch (_) {
    nuevo = const DivisasError('Ocurrio un error inesperado.');
  }
  if (idPeticion != _peticionActual) return;   // respuesta obsoleta
  _estado = nuevo;
  _cargando = false;
  notifyListeners();
}
```

### UI: switch exhaustivo sobre el estado

```
// lib/ui/screens/divisas_screen.dart
final provider = context.watch<DivisasProvider>();

RefreshIndicator(
  onRefresh: () => provider.cargar(refrescar: true),          // pull-to-refresh
  child: switch (provider.estado) {                            // exhaustivo
    DivisasCargando() => const _VistaCargando(),               // esqueletos
    DivisasExito(:final datos) => ListView.builder(
      physics: const AlwaysScrollableScrollPhysics(),
      itemCount: datos.tasas.length,
      itemBuilder: (_, i) => TasaCard(
        tasa: datos.tasas[i], base: datos.base, monto: provider.monto),
    ),
    DivisasError(:final mensaje, :final sinConexion) => _VistaError(
      mensaje: mensaje,
      sinConexion: sinConexion,
      alReintentar: () => provider.cargar(),
    ),
  },
)
```

# Sección IV: Evidencias de pruebas (QA) y enlace al repositorio

## 4.1 Repositorio público

**URL:** `https://github.com/jeffvera281996/catalogo-productos-flutter-p3`

El trabajo de esta parte está en la rama `feature/api-integration`, integrada en `develop` y `main` con `--no-ff` y etiquetada `v3.0.0`. El README incluye la sección "API consumida (Parte 3)" con el endpoint, el JSON de ejemplo y los pasos para ejecutar la app.

| Commit | Contenido |
|---|---|
| **feat(api)** | Modelo TasasCambio y fuente remota Frankfurter |
| **feat(estado)** | DivisasProvider con estados Cargando, Éxito y Error |
| **feat(ui)** | Pantalla Divisas con tarjetas, conversor y pull-to-refresh |
| **test(api)** | Mapeo JSON y transiciones de estado con MockClient |
| **docs** | README e informe de la Parte 3 |
| **chore(release)** | NexoShop 3.0.0 |

## 4.2 Casos de prueba

| Caso | Pasos | Resultado esperado |
|---|---|---|
| **Cargando** | Abrir Divisas con red lenta (emulador: Cellular → GSM) | Tarjetas esqueleto animadas |
| **Éxito** | Abrir Divisas con internet | Lista de 30 monedas y "Tasas del BCE al dd/mm/aaaa" |
| **Pull-to-refresh** | Deslizar la lista hacia abajo | Indicador circular; la lista se mantiene y se actualiza |
| **Cambio de base** | Tocar EUR, GBP o MXN | Nueva petición; el monto se expresa en la nueva base |
| **Error de red** | Modo avión → tocar Actualizar | Wi-Fi tachado, "No pudimos conectarnos" y Reintentar |
| **Recuperación** | Quitar modo avión → Reintentar | Vuelve al estado Éxito |
| **Pruebas unitarias** | flutter test test/divisas_test.dart | Mapeo, ida y vuelta JSON, Éxito, sin red, HTTP 500, JSON inválido y refresco |

## 4.3 Capturas de funcionamiento

*Reemplazar cada recuadro por la captura real del emulador o dispositivo.*

**Captura 1: Cargando**
Pegar aquí: Tarjetas esqueleto

*Figura 2. Estado Cargando.*

**Captura 2: Éxito**
Pegar aquí: Lista con tasas

*Figura 3. Estado Éxito con datos de la API.*

**Captura 3: Error de red**
Pegar aquí: Modo avión activado

*Figura 4. Manejo del error de red.*

**Captura 4: Pull-to-refresh**
Pegar aquí: Indicador circular sobre la lista

*Figura 5. Actualización manual.*

**Captura 5: Conversión en otra base**
Pegar aquí: Por ejemplo, base EUR y monto 50

*Figura 6. Cambio de moneda base.*

# Sección V: Conclusiones

## 5.1 La asincronía protege la experiencia

Una petición HTTP puede tardar milisegundos o varios segundos, según la red. Con async/await y Future, la interfaz sigue respondiendo mientras se espera la respuesta: el usuario puede desplazarse, escribir un monto o volver atrás. Si la petición se hiciera de forma bloqueante, la app se congelaría y Android podría mostrar el aviso "La aplicación no responde". El timeout de 8 segundos cierra el ciclo: ninguna espera es infinita.

## 5.2 El error de red es un estado normal, no una excepción rara

En un teléfono la conexión cambia todo el tiempo: túneles, ascensores, Wi-Fi sin salida a internet, datos móviles saturados. Por eso el error se modeló como un estado más, al mismo nivel que Cargando y Éxito. Al usar una jerarquía sealed, el compilador no deja que la pantalla ignore ese caso. Traducir los errores técnicos (SocketException, código 500, JSON inválido) a mensajes claros, con una acción para recuperarse (Reintentar o deslizar), convierte un fallo en una situación que el usuario entiende y puede resolver.

## 5.3 Desacoplar facilita probar y cambiar

Como la pantalla solo conoce el Provider, se pudieron probar todos los estados sin red real, inyectando un MockClient que simula respuestas 200, 500, JSON roto o falta de conexión. El mismo diseño permitiría cambiar Frankfurter por otra API, o el paquete http por dio, sin tocar la interfaz.

## 5.4 Conclusión general

Integrar una API pública parece sencillo: una línea con http.get. Lo que distingue a una app profesional es lo que rodea esa línea: modelos tipados que validan los datos, estados explícitos para cada resultado posible, peticiones que no se pisan entre sí y una interfaz que siempre le dice al usuario qué está pasando. Esta parte completa a NexoShop como una app conectada que funciona bien tanto con buena señal como sin ella.